

Merchant Risk Intelligence System

LangGraph + CrewAI Combined — Vibe Coding Challenge

30 minutes / Individual / LLM-assisted coding allowed

Mission Brief

Sigma DataTech processes 4 million transactions per day. The fraud team gets 200+ alerts daily but only has time to investigate 20. They need an intelligent triage system that:

- Quickly screens every flagged merchant (fast, no LLM needed)
- Deep-investigates only the high-risk ones using a 3-agent crew
- Produces a structured risk verdict they can act on immediately

You are building that system in 30 minutes using LangGraph as the controller and CrewAI as the investigation engine.

Why This Needs Both Frameworks

Layer	Framework	Why
Routing & Control	LangGraph	Decides: screen only vs. full investigation. Conditional edge based on risk score.
Deep Investigation	CrewAI	3 agents with different expertise collaborate to produce a rich verdict.
Data Access	DuckDB (both)	Same sigma_platform.duckdb used in all labs — no new setup needed.

Architecture

The graph has 3 nodes. CrewAI lives entirely inside the `investigate_node`:

```
graph TD
    START --> screen_node
    screen_node["screen_node (LangGraph)"] --> investigate_node
    investigate_node["investigate_node (CrewAI)"] --> END
```

Pulls txn stats from DuckDB.
Calculates risk_score (0-100).

```

    No LLM. Pure Python.
      |
      +-----+-----+
      |           |           |
risk_score < 50   risk_score >= 50
      |           |           |
  clear_node    investigate_node  <-- CrewAI crew runs here
Writes 'CLEARED' 3 agents:
verdict to JSON. Scout -> pulls detailed txn data
                  Analyst-> finds anomaly patterns
                  Reporter-> writes risk verdict
      |           |           |
      +-----+-----+
              |
              END

```

What To Build

File to create

Create a single file:

```
day10/lab/5_casestudy_risk_agent.py
```

Step 1 — Define the LangGraph State (5 min)

Create a TypedDict with these fields:

```

merchant_id : str      # input - the merchant to investigate
risk_score  : int      # set by screen_node (0-100)
txn_summary : str      # brief stats string from screen_node
verdict     : str      # final output - CLEARED or investigation result

```

Step 2 — Build screen_node (8 min)

Pure Python, no LLM. Query sigma_platform.duckdb for the merchant and compute a risk score:

- Transaction count in last 30 days
- Average transaction amount
- Null rate on merchant_id column
- Your scoring logic: high volume + low avg amount + any nulls = high risk

Return: { risk_score: int, txn_summary: str }

Step 3 — Build investigate_node with CrewAI (12 min)

This node creates and runs a 3-agent crew. Each agent gets state['txn_summary'] in their task description:

Agent	Role / Goal	Task	Model
Scout	Data Retriever — pull raw evidence	Get top 10 txns for this merchant, identify patterns	Nova Lite
Analyst	Pattern Detective — find anomalies	Analyse Scout's data, flag suspicious signals	Nova Lite
Reporter	Risk Officer — write the verdict	Summarise findings as RISK LEVEL + 3-line explanation	Nova Pro

Return: { verdict: str — the crew's final report }

Step 4 — Build clear_node (2 min)

Simple — just set verdict = 'CLEARED: risk_score below threshold' and return.

Step 5 — Wire the graph (3 min)

```
g = StateGraph(State)
g.add_node('screen', screen_node)
g.add_node('investigate', investigate_node)
g.add_node('clear', clear_node)
g.set_entry_point('screen')
g.add_conditional_edges('screen', route_by_risk,
    {'investigate': 'investigate', 'clear': 'clear'})
g.add_edge('investigate', END)
g.add_edge('clear', END)
app = g.compile()
```

Write route_by_risk(state): returns 'investigate' if state['risk_score'] >= 50, else 'clear'.

Step 6 — Run and save output (2 min)

```
result = app.invoke({'merchant_id': 'M1042', 'risk_score': 0,
    'txn_summary': '', 'verdict': ''})

import json
with open('agent_outputs/risk_verdict.json', 'w') as f:
    json.dump(result, f, indent=2)
print(result['verdict'])
```

Expected Output

If risk_score < 50 — CLEARED path:

```
{
  "merchant_id": "M1042",
  "risk_score": 32,
  "txn_summary": "428 txns, avg $85.20, 0 nulls",
  "verdict": "CLEARED: risk_score 32 below threshold"
}
```

If risk_score >= 50 — Investigation path:

```
{
  "merchant_id": "M2187",
  "risk_score": 74,
  "txn_summary": "1842 txns, avg $8.40, 23 nulls",
  "verdict": "RISK LEVEL: HIGH\n
    Scout found 1842 transactions with unusually low avg $8.40.\n
    Analyst flagged micro-transaction pattern — possible card testing.\n
    Recommend: freeze merchant and escalate to fraud team."
}
```

Hints — Read Only If Stuck

Stuck on...	Hint
DuckDB connection	<code>conn = duckdb.connect('day10/lab/sigma_platform.duckdb', read_only=True)</code>
CrewAI LLM setup	<code>llm = LLM(model='bedrock/amazon.nova-lite-v1:0', region_name='us-east-1')</code>
Passing data to crew	Put state['txn_summary'] directly in the task description string
Getting crew result	<code>result = crew.kickoff()</code> — <code>result.raw</code> gives you the string output
Risk score logic	<code>score = 0; if txn_count > 1000: score += 40; if avg_amount < 20: score += 40; if nulls > 0: score += 20</code>
LLM prompt for crew	Tell your LLM: 'write CrewAI code with 3 agents, context= chain, Process.sequential'

Success Criteria

#	Criterion	Points
1	Script runs without errors on both a low-risk and high-risk merchant	30
2	Low-risk merchant takes the CLEAR path — no CrewAI crew is triggered	20
3	High-risk merchant triggers all 3 agents — Scout, Analyst, Reporter all visible in output	30
4	risk_verdict.json written to agent_outputs/ with all 4 state fields populated	20
B	BONUS: Add a 4th agent — Recommender — that suggests one specific SQL fix to clean the merchant data	+20

Debrief — Answer Before You Leave

Be ready to answer one of these when showing your output:

- Why does screen_node NOT use an LLM? What would break if it did?
- The clear_node and investigate_node both end at END. Why is that fine?
- What happens if you remove context= from the Analyst's task? Try it.
- Could you replace the CrewAI crew with a single LangGraph node that calls Bedrock 3 times? What would you lose?